Kitchnos — Product Requirements Document (PRD)

Nov 9, 2026 · @Gaurav

Field	Value
Product	Kitchnos (working name: Kitchen AI)
Document	Product Requirements Document, v1.0
Status	Draft for review
Scope	V1 in full detail; V1.5–V3 at outline level
Related	Kitchnos BRD · Architecture CSV pack · GitHub backlog (user stories trace to the IDs below)

1. Product summary

Kitchnos is a mobile kitchen companion that knows the household's kitchen, decides what to cook, cooks with the user step by step (by touch or by voice, in their language) and learns from every meal. Every feature reads from and writes back to one **Kitchen Brain**: pantry, household members, taste memories, recipes cooked and the live cooking session.

The **Voice Cook** is the assistant cook inside the app. It is not a separate product or a separate brain: it talks to the same services the screens use, so it already knows what is in the kitchen, who is eating, how spicy they like it and which step the user is on.

The first magic moment (V1 acceptance loop):

1. Populate a small kitchen by talking ("do kilo aloo, pyaaz, besan, dahi hai").
2. Say what you're craving.
3. Get one pick that uses what you have, with a one-line reason.
4. Start Cook With Me, hands-free.
5. Ask a natural question mid-step and get an answer about *this* step.
6. Finish the meal.
7. Say "agli baar lahsun zyada."
8. See that become a memory on the You tab.
9. See the pantry quantities update.

2. Goals and non-goals

Goals (V1)

1. Remove the daily "what should I cook?" decision with one trustworthy pick (BR-02).
2. Keep a living pantry that stays accurate with little effort (BR-01, BR-03, BR-04).
3. Make anyone a capable cook, including first-timers and people who can't read the app, through step-by-step guidance and the Voice Cook (BR-05, BR-06, BR-07).
4. Cook from what you have: adapt any dish to missing items, equipment and family needs (BR-08, BR-09).
5. Get smarter after every meal, under the user's control (BR-10).

Non-goals

- A general chatbot. The assistant only talks about cooking, food, the kitchen and shopping.
- A recipe social network or creator platform.
- Calorie counting, medical diets or diagnosis.
- Restaurant ordering or food delivery.
- Smart-appliance control (may come later through partners).
- Buying anything without explicit approval of items and total.

3. Personas

Five personas cover the BRD segments; Sunita and Kamla are the launch personas every V1 screen is tested against.

Persona	Context	Needs	Frustrations today	What Kitchnos must do
Sunita, 38, Jaipur (launch)	Cooks three meals a day for husband, two kids and mother-in-law; Hindi and Hinglish; mid-range Android	Variety that suits everyone; quick morning decisions; less waste	Same rotation every week; kids reject spicy food; vegetables spoil	Daily pick with family plates; Use Soon; voice while cooking

Persona	Context	Needs	Frustrations today	What Kitchnos must do
Kamla, 45, village near Sriganganagar (launch pilot)	Reads little; shares her son's phone; Marwari-influenced Hindi; gas stove and pressure cooker	Make favourite dishes and new variations from what's at home, by talking	Apps are in English and full of text; no one to ask mid-cooking	Voice-only onboarding and cooking; her measures (katori, mutthi, seeti); patient pace
Arjun, 28, Gurugram	Cooks for himself and partner on weeknights; Hinglish; iPhone; air fryer	Healthy meals in 30 minutes; less ordering in	Buys vegetables that rot; recipes assume a full masala dabba	Pantry-aware quick picks; air-fryer adaptations; weekly plate
Riya, 21, Pune hostel	Beginner, away from home; induction plate and kettle	Learn basics; recreate home food	Recipes say "till done"; one failure and she orders in	Sensory cues, photo checkpoints, Recipe Rescue, technique learning
The Mehta household (V2)	Planner works; a domestic cook makes the food; the cook speaks Bhojpuri	Plan menus and hand them over	Daily calls to explain the menu	Cook handoff by voice note in the cook's language

4. Key user journeys

Every journey works by touch and by voice; the voice column shows what the user can say instead of tapping. Step-by-step data flows for each journey are in `09_data_flows.csv`.

Journey	Flow	Voice path	Success signal	Release
J1 First setup	Language → who eats here → spice and diet → consent → "what's in your kitchen?" → Home with first pick	"Hindi mein. Hum chaar log hain, teekha pasand hai."	First pick shown in under 2 minutes	V1
J2 I'm hungry	Home → craving → ranked picks from pantry → recipe → Cook With Me	"Kuch halka aur chatpata"	Pick accepted and cooking started	V1
J3 Tonight, decided	4 pm nudge → one pick + two alternates → Cook this	"Haan, yahi banate hain"	Accepted without browsing	V1
J4 Voice Cook	Start → hands-free steps → timers → questions → rescue → finish	"Agla batao", "kitna namak?", "ruko"	Session finished with no taps after start	V1
J5 Cooking problem	Something's wrong → state-aware fix → continue	"Namak zyada ho gaya!"	Session completed after a rescue	V1
J6 After dinner	Complete → feedback → memory → inventory deduction review → leftovers	"Bahut badhiya, agli baar lahsun zyada"	Memory saved; inventory updated	V1 (leftovers V2)
J7 Grocery haul	Speak or type the haul → parse → confirm → storage → shelf life	"Do kilo aloo aur ek dabba dahi laayi hoon"	Haul added in under 10 seconds	V1 (scan V1.5)
J8 Food at risk	Use Soon item → waste-aware recipes → cook → update	"Palak kharab hone wala hai, kya banaun?"	Item cooked before expiry	V1

Journey	Flow	Voice path	Success signal	Release
J9 Missing items	Recipe → missing ingredients → shopping list → share or provider cart → approval	"Tamatar list mein daal do"	Item on list; bought and added	V1 (cart V2)
J10 Restock	Prediction → ask → cart → approval	"Haan, atta mangwa do"	Approved order placed	V2
J11 WhatsApp voice note	Voice note with pantry or question → voice reply → deep link to Cook	Any of the above, on WhatsApp	Reply within 5 seconds	V2

5. Functional requirements: onboarding and Home

Priority: P0 = must ship in the stated release, P1 = should, P2 = could. Each ID maps to user stories in `14_user_stories.csv`.

5.1 Onboarding (ONB)

ID	Requirement	Priority	Release	Acceptance criteria
ONB-01	Start without email or password: "Try now" creates an anonymous account; phone OTP upgrades it later without data loss	P0	V1	Home reached in ⩽ 60 s on first launch; anonymous data survives upgrade
ONB-02	Choose language by voice or tap (Hindi, Hinglish, English, launch regional language)	P0	V1	All later screens and voice replies use it; changeable in You
ONB-03	Household setup in three steps: who eats here, spice level, diet (veg, eggetarian, non-veg, Jain)	P0	V1	Creates household + members + taste profiles
ONB-04	Plain-language consent for voice processing, memory and analytics, shown and spoken	P0	V1	Consent stored per member with timestamp; app usable with voice declined

ID	Requirement	Priority	Release	Acceptance criteria
ONB-05	Voice-only onboarding path for low-literacy users	P0	V1	A first-time user completes setup without reading, in field test
ONB-06	Kitchen equipment checklist (pressure cooker, kadai, tawa, air fryer, OTG, microwave, induction, chulha)	P1	V1	Recipes adapt to equipment owned
ONB-07	"What did you cook yesterday?" quick start seeds taste and likely pantry	P1	V1	Seeds at least 5 pantry items as unconfirmed

5.2 Home and recommendations (HOME)

ID	Requirement	Priority	Release	Acceptance criteria
HOME-01	**Tonight, decided:** one hero pick and two alternates for the current meal slot, each with a one-line reason	P0	V1	Reason names at least one real factor (item used, time, taste, member)
HOME-02	Ranking uses pantry coverage, Use Soon items, taste profiles, who's eating, recent meals (no repeat within 3 days by default), time available, equipment and calendar (fasts, festivals)	P0	V1	Score breakdown stored per recommendation for learning and debugging
HOME-03	Craving input by voice or text with quick chips: Comforting, Under 20 min, High protein, Kids friendly, Light	P0	V1	Free text in any supported language understood
HOME-04	Use Soon strip showing items at risk, tappable into waste-aware recipes	P0	V1	Items sorted by days left × quantity

ID	Requirement	Priority	Release	Acceptance criteria
HOME-05	Meal-slot awareness (breakfast, lunch, tiffin, dinner, snack) by time of day	P1	V1	Slot switchable by tap or voice
HOME-06	Dismiss with a reason (not today, don't like, no time, missing item)	P1	V1	Reason feeds learning; "don't like" proposes a memory
HOME-07	Daily pick notification at a chosen time (default 16:00), max one per day	P1	V1	Opt-in; quiet hours respected
HOME-08	Designed loading ("Thinking about tonight…"), empty kitchen, offline and error states	P0	V1	Each state has copy, illustration and a recovery action

6. Functional requirements: Kitchen and pantry tracking

The pantry is a **probable** model, not a strict list: every item carries a confidence that decays over time and rises when the user confirms, cooks or buys. The app asks only the questions that change tonight's plan.

6.1 Kitchen (KIT)

ID	Requirement	Priority	Release	Acceptance criteria
KIT-01	Locations: Fridge, Pantry, Freezer, Staples, Ready-made (homemade components and leftovers)	P0	V1	Each item belongs to one location; moves are logged
KIT-02	Item model: name, quantity and unit (metric and local units), location, state (raw, opened, cooked, homemade, leftover), expiry (actual or estimated), confidence	P0	V1	Estimated expiry visibly marked as estimated

ID	Requirement	Priority	Release	Acceptance criteria
KIT-03	Add by voice or text: "2 kilo aloo, ek dozen ande, aadha kilo paneer" → parse → confirm → commit	P0	V1	90% of items parsed correctly on the test set; nothing committed without confirmation
KIT-04	Edit, mark finished or discard with one tap or one sentence	P0	V1	Every change writes an inventory event
KIT-05	Automatic deduction after cooking, shown as a reviewable summary ("Used 200 g paneer, 2 onions")	P0	V1	User can correct any line before it commits
KIT-06	Staples tracked as Full / Low / Out instead of exact quantities (atta, rice, dal, oil, salt, spices)	P0	V1	Low or Out adds to the shopping list on confirmation
KIT-07	Homemade components (chutneys, pickles, masala bases, boiled chana) as first-class items with flavour roles: rich, heat, acid, crunch, starch	P0	V1	Components appear in picks and "Why this works"
KIT-08	Confidence decay and targeted checks: at most one check question a day unless the user asks	P1	V1	Only asked when the answer changes the current pick
KIT-09	Storage advice and shelf-life estimate per ingredient and location	P1	V1	Shown on item detail
KIT-10	Household-shared kitchen synced live across members' devices	P1	V1	A change on one phone shows on another within 3 s

6.2 Input methods (INP)

ID	Requirement	Priority	Release	Acceptance criteria
INP-01	Barcode scan → product lookup → prefilled item; rapid sequential scanning	P0	V1.5	Known barcode added in ⩽ 2 s

ID	Requirement	Priority	Release	Acceptance criteria
INP-02	Camera scan of fridge or shelf → detected items with confidence → "I found 6 items — did I get it right?" → commit	P0	V1.5	No item committed without confirmation
INP-03	Receipt photo → line items → normalised ingredients → bulk confirm	P1	V1.5	Line-item accuracy ⩾ 85% on test receipts
INP-04	Grocery order sync from official provider integrations	P1	V2	Expected items added as unconfirmed until delivery

7. Functional requirements: recipes, adaptation and household plates

Recipes are **generated plans built on curated foundations**: a human-checked base dish plus variation rules, which the AI adapts to the user's real kitchen. The model never invents a dish without passing the safety rules.

7.1 Recipes (REC)

ID	Requirement	Priority	Release	Acceptance criteria
REC-01	Curated base of 300 dishes at launch: local names, base servings, steps, sensory cues, techniques, reference photos, variation rules	P0	V1	Every dish cooked and checked by the culinary lead
REC-02	Pantry match: "You have 9 of 10" with the missing item, substitutes and Add to list	P0	V1	Coverage computed from confident items only
REC-03	Adapt a recipe: substitute, omit, scale servings, change spice, swap equipment, change diet; save as a personal variant	P0	V1	Adapted plan keeps step count sane and safety rules

ID	Requirement	Priority	Release	Acceptance criteria
REC-04	Units in the user's system: grams and ml, or katori, chammach, mutthi, cup; pressure-cooker whistles as time	P0	V1	Gram-per-measure conversions stored per ingredient, set and tested by the culinary lead
REC-05	"Why this works": flavour balance of rich, heat, crunch, acid, with one sentence	P1	V1	Shown on recipe detail
REC-06	Invent a combination from what's here (wireframe's "dish worth repeating"), using flavour logic and components	P1	V1	Passes safety rules; marked as a new creation
REC-07	Save, favourite and collections	P1	V1	Synced to household
REC-08	Search recipes by name in any supported language and script	P1	V1	"rajma", "राजमा" and "kidney bean curry" return the same dish

7.2 Household plates (HH)

ID	Requirement	Priority	Release	Acceptance criteria
HH-01	Household members with spice level, diet, dislikes, age group (child, adult, elder) and free-text notes such as "less oil"	P0	V1	Up to 6 members in V1
HH-02	**Family plates:** one pot, per-member branches inside the steps ("take out the kids' portion before adding chilli")	P0	V1	Branch steps appear at the right step, by member name
HH-03	"Who's eating today?" sets servings and filters picks	P1	V1	Defaults to everyone
HH-04	Member roles: owner, member, viewer; cook role in V2	P1	V1	Enforced by row-level security

ID	Requirement	Priority	Release	Acceptance criteria
HH-05	Cook handoff: send the day's menu to the domestic cook as a voice note in her language	P1	V2	Delivered over WhatsApp with consent

8. Functional requirements: Cook With Me and Recipe Rescue

Cook With Me is a **stateful sous-chef**: it always knows the recipe, step, timers, substitutions made and anything that went wrong, whether the user is tapping or talking.

8.1 Cook With Me (COOK)

ID	Requirement	Priority	Release	Acceptance criteria
COOK-01	One main instruction per screen in large type, with step progress	P0	V1	Readable at arm's length (≥ 28 pt instruction text)
COOK-02	First-class timers: several at once, labelled, survive backgrounding, loud alarm	P0	V1	Timer fires correctly with the app closed
COOK-03	Pressure-cooker whistle counting by tap or voice ("ek seeti hui"); automatic audio detection later	P1	V1 (audio V2)	Count shown and spoken
COOK-04	Sensory cue and reference photo per step ("It should look like this")	P0	V1	Every curated step has a cue; photos for the top 50 dishes
COOK-05	Photo checkpoint: user shows the pan; AI assesses and adapts ("1 more minute for deeper colour")	P1	V1	Answer within 4 s; uncertainty stated
COOK-06	Ask anything mid-step without losing state	P0	V1	Answer refers to the current step and ingredients
COOK-07	"I changed something" updates the remaining steps; offer to remember the change	P0	V1	Changes logged as session events

ID	Requirement	Priority	Release	Acceptance criteria
COOK-08	Offline cooking: a session bundle (steps, photos, cached audio, timers) downloads at start	P0	V1	Airplane mode after start still completes the dish
COOK-09	Screen stays awake; touch targets ⩾ 56 dp; works with wet or floury hands (large swipe areas)	P0	V1	Passes device test checklist
COOK-10	Completion: celebrate, collect feedback, review inventory deduction, offer to save leftovers	P0	V1	All four in one sheet, skippable
COOK-11	Resume an interrupted session from the same step	P1	V1	Resumes within 24 h

8.2 Recipe Rescue (RES)

ID	Requirement	Priority	Release	Acceptance criteria
RES-01	"Something's wrong" always one tap or one sentence away: too salty, burning, too watery, too thick, too spicy, undercooked, missing ingredient	P0	V1	Reachable from every cook step
RES-02	Fixes are state-aware: use the current step, dish and what's in the pantry	P0	V1	Max 3 short steps; then "Done, continue"
RES-03	Food-safety guards: undercooked meat, raw-egg sauces, reheated rice, spoiled items	P0	V1	Safety advice overrides convenience and is never softened by the model

9. Functional requirements: Voice Cook assistant

The Voice Cook is the assistant cook inside Kitchnos: it helps each person in their way, in their language, personalised through the pantry and taste memory. Technically it is a voice front-end to the Kitchen Brain: it reads a context packet every turn and acts only through the same services the screens use.

9.1 Requirements (VOICE)

ID	Requirement	Priority	Release	Acceptance criteria
VOICE-01	Voice on every tab: Home (craving, picks), Kitchen (add, edit, ask), Cook (full control), You (memories)	P0	V1	One mic button, same placement everywhere
VOICE-02	Languages at launch: Hindi, Hinglish, English and one regional language; replies in the language the user speaks, including mixed speech	P0	V1	Word error rate on the kitchen test set ⩽ 15% per language
VOICE-03	Hands-free Cook mode: after "shuru karo" it keeps listening, with voice detection filtering kitchen noise; no wake word while a session is open	P0	V1	No taps needed from start to finish
VOICE-04	Barge-in: the user can interrupt the assistant at any moment ("ruko", "phir se")	P0	V1	Assistant audio stops within 300 ms
VOICE-05	Latency: end of user speech to first assistant audio	P0	V1	p50 ⩽ 1.0 s, p95 ⩽ 1.8 s on 4G; logged per turn
VOICE-06	Fast-path commands handled without the language model: next, back, repeat, slower, pause, set/check timer, done	P0	V1	Fast-path reply ⩽ 400 ms; work offline in Cook mode
VOICE-07	Agent acts through tools mapped to services (full list: `07_voice_agent_tools.csv`)	P0	V1	No direct database writes from the agent

ID	Requirement	Priority	Release	Acceptance criteria
VOICE-08	Spoken confirmation before any write: pantry changes, memories, list items, timers over 30 min ("Do kilo aloo jod diye, theek?")	P0	V1	Writes without confirmation = 0 in audit
VOICE-09	Personal voice behaviour from the voice profile: units, pace, detail by skill level, voice choice	P0	V1	Beginner gets sensory cues; expert gets short prompts
VOICE-10	Noise robustness: ignore cooker whistles, frying, TV, children; ask to repeat when confidence is low	P0	V1	False triggers ⩽ 1 per 15-min session in field tests
VOICE-11	Privacy: raw audio never stored by default; transcripts kept 30 days if consented, for quality and learning	P0	V1	Verified by storage audit
VOICE-12	Offline fallback: cached step audio plays and fast-path commands work when the network drops	P1	V1	Session continues in airplane mode
VOICE-13	Shared phone: switch speaker by name ("Main Sunita hoon") to load her taste and voice profile	P1	V1	Switch takes one sentence
VOICE-14	Cost controls: voice-detection gating of speech-to-text, cached step audio, Haiku for turns, idle disconnect during long timers	P0	V1	Variable voice cost ⩽ ₹5 per 15-min session
VOICE-15	WhatsApp voice-note channel on the same Kitchen Brain	P1	V2	Reply within 5 s
VOICE-16	Phone-call channel for keypad and borrowed phones	P2	V3	Same tools, same memories

9.2 How one voice turn works

1. **LiveKit WebRTC** carries her audio from the app to the voice agent worker (LiveKit Agents, Python) in an India region, with echo cancellation.
2. **Voice detection** (Silero, in the agent) decides when she is speaking; only those segments go to speech-to-text, so kitchen noise between turns is never transcribed or billed.

3. **Speech-to-text:** Sarvam streaming recognition returns the words, the detected language and a confidence.
4. **Fast path:** if the words match a cooking command (next, repeat, timer…), the agent acts immediately and answers from cached audio.
5. **Otherwise the agent builds the context packet** (`get_voice_context` RPC: voice profile, household, pantry with confidence, taste, live session state, recent meals) and calls **Claude Haiku 4.5** with the tools.
6. **Tools run** through Supabase RPCs and Edge Functions under the user's identity, so row-level security still applies.
7. **Text-to-speech:** Sarvam Bulbul streams audio back sentence by sentence, so speech starts before the full reply is written.
8. **Session events** (command, question, answer, tool call, latency, cost) are written for learning and the cost dashboard; audio is discarded.

9.3 Conversation rules

- Offer two choices, never ten, and end with one simple question.
- Repeat back to confirm ("Theek hai, besan wali sabzi. Shuru karein?").
- One step at a time, with a sensory cue and the user's units ("do mutthi besan", "teen seeti").
- Never ask for something the Kitchen Brain already knows.
- Separate today from forever: "aaj kam teekha" changes this session; "hamesha kam teekha" proposes a memory.
- Stay on cooking, food, the kitchen and shopping; politely decline everything else.
- Safety advice is never softened, shortened or skipped.

Example (Kamla, Hindi):

> **Kamla:** Ghar pe aloo, pyaaz, thoda besan aur dahi hai. **Kitchnos:** Badhiya! Aaj aloo pyaaz ke pakode ya besan wali aloo sabzi bana sakti ho. Kaunsi? **Kamla:** Sabzi. Tamatar nahi hai. **Kitchnos:** Koi baat nahi, tamatar ki jagah thoda dahi daal denge. Pehle do pyaaz kaat lo. Ho jaye to bolna.

10. Functional requirements: taste learning and memory

Learning is continuous, explainable and under the user's control. The blueprint's memory rules are binding: explicit feedback outranks assumptions, behaviour is evidence not certainty, temporary instructions stay temporary, and every important memory keeps its source.

ID	Requirement	Priority	Release	Acceptance criteria
LRN-01	One-tap feedback after cooking (Loved it, Good, Meh) plus chips (Too spicy, More garlic, Make again) and optional voice	P0	V1	Feedback rate ⩾ 50% of completed sessions
LRN-02	Memory record: statement, category, scope (permanent or temporary), source (explicit, observed, correction), confidence, evidence count, last confirmed	P0	V1	Every memory shows where it came from
LRN-03	Temporary instructions apply to the current session only unless the user promotes them	P0	V1	"Aaj kam teekha" never changes the taste profile
LRN-04	Corrections override conflicting inferences immediately ("Actually I hate mushrooms")	P0	V1	Conflicting memory marked superseded
LRN-05	"What I've learned" on You, with Edit · Keep · Forget, also by voice ("yeh bhool jao")	P0	V1	Forget deletes the memory and anything derived only from it
LRN-06	Taste profile per member updates from feedback: spice, sweetness, richness, sourness, textures, time preference	P0	V1	Visible changes after 3 meals
LRN-07	Ingredient likes and dislikes per member with confidence	P0	V1	Disliked ingredients ranked down, never silently removed from shared dishes
LRN-08	Picks visibly reflect memories ("less spicy, as you asked")	P1	V1	At least one memory referenced in reasons when relevant
LRN-09	Technique ladder: skills per member (tadka, bhuna, dum…) that unlock dishes	P2	V2	Skill level adjusts Voice Cook detail

ID	Requirement	Priority	Release	Acceptance criteria
LRN-10	Observed patterns (cooks under 30 minutes on weekdays) become suggestions to confirm, not silent rules	P1	V2	User confirms before it becomes permanent

11. Functional requirements: waste, health, shopping and grocery agent

11.1 Use Soon and leftovers (LEFT)

ID	Requirement	Priority	Release	Acceptance criteria
LEFT-01	Rank recipes by how much at-risk food they use	P0	V1	Use Soon items appear in reasons ("uses your spinach today")
LEFT-02	Save cooked food as a leftover at completion, tracked apart from unopened stock	P1	V2	Leftover appears in Ready-made with a use-by
LEFT-03	Leftover transformations (rice → lemon rice; roti → churi; chicken → wrap)	P1	V2	Suggested within the leftover's safe window
LEFT-04	Gentle reminders, never more than one waste nudge a day	P1	V1	Counted in the daily notification cap

11.2 Health nudges (HLTH)

ID	Requirement	Priority	Release	Acceptance criteria
HLTH-01	One balance nudge per meal inside the dish ("add a katori of curd"), aligned with ICMR-NIN guidance	P1	V1.5	Never more than one per meal; dismissable
HLTH-02	Weekly plate: food groups cooked this week (dal, sabzi, greens, fruit, dairy, protein), no calorie numbers	P1	V1.5	Computed from sessions, not manual logging

ID	Requirement	Priority	Release	Acceptance criteria
HLTH-03	Member-aware nudges for children and elders without medical claims	P2	V2	Reviewed by a nutritionist before release

11.3 Shopping (SHOP)

ID	Requirement	Priority	Release	Acceptance criteria
SHOP-01	Add missing recipe ingredients to the list in one tap or sentence	P0	V1	Linked to the recipe that needed it
SHOP-02	Add Low and Out staples on confirmation	P0	V1	No silent additions
SHOP-03	Group the list by shop section; share as WhatsApp text	P1	V1	Shared list readable without the app
SHOP-04	Mark bought → add to pantry with estimated expiry	P0	V1	One tap per item or "sab le aaye"

11.4 Grocery agent (GROC)

ID	Requirement	Priority	Release	Acceptance criteria
GROC-01	Provider-independent actions: find product, get cart, add, update, remove, checkout, via official integrations only	P0	V2	Each provider behind one adapter interface
GROC-02	Approval screen with items, quantities, provider and total; spoken summary in voice mode	P0	V2	No order without an explicit yes
GROC-03	Predictive restocking from purchase patterns ("atta usually runs out in 12 days")	P1	V2	Prediction shown as a question, not an order
GROC-04	Provider comparison where integrations allow	P2	V3	Same items priced side by side

11.5 Notifications (NOTIF)

ID	Requirement	Priority	Release	Acceptance criteria
NOTIF-01	At most one proactive notification a day by default (pick, Use Soon or restock), with quiet hours	P0	V1	Cap enforced server-side
NOTIF-02	Timer alarms and cooking reminders are exempt from the cap	P0	V1	Alarms work with the app closed

12. Non-functional requirements

ID	Area	Requirement	Target
NFR-01	Performance	Cold start to Home on a 3 GB RAM Android	⩽ 3 s
NFR-02	Performance	Home pick ready after open (cached pick shown instantly, refreshed in background)	⩽ 2.5 s fresh; instant cached
NFR-03	Voice	Voice turn latency (VOICE-05)	p50 ⩽ 1.0 s, p95 ⩽ 1.8 s
NFR-04	Offline	Cook mode, timers, cached audio and fast-path commands work offline after session start	100% of curated dishes
NFR-05	Offline	Pantry edits made offline sync when back online, last-write-wins with event log	No data loss in tests
NFR-06	Device	Android 9+ and iOS 16+; install size	⩽ 40 MB download
NFR-07	Network	Usable on 3G-class networks; images served as small thumbnails first	Home ⩽ 300 KB per load
NFR-08	Languages	UI strings and voice in Hindi, Hinglish, English + 1 regional at launch; all strings externalised	No hard-coded copy

ID	Area	Requirement	Target
NFR-09	Accessibility	WCAG 2.1 AA contrast; dynamic text to 200%; TalkBack and VoiceOver labels; touch targets ≥ 48 dp (≥ 56 dp in Cook)	Audit passes
NFR-10	Privacy	Consent per purpose under India's DPDP Act 2023; data export and account deletion in-app	Delete completes ≤ 30 days, data gone from backups per policy
NFR-11	Privacy	No raw audio stored by default; transcripts 30 days; images (scans, checkpoints) 90 days unless saved	Retention jobs verified
NFR-12	Security	Row-level security on every table; service keys never in the app; secrets in Supabase and EAS secret stores	Security review before beta
NFR-13	Security	Data hosted in India (Supabase Mumbai region; India-region media servers where available)	Configured at project creation
NFR-14	Reliability	Backend availability	99.5% monthly
NFR-15	Observability	Crash reporting, per-turn voice latency and cost, AI request logs with model and tokens	Dashboards before beta
NFR-16	Cost	Variable cost per 15-min voice session (BR-19)	≤ ₹5
NFR-17	AI quality	Recipe adaptation and rescue evaluated against a fixed test set before every model or prompt change	No regression merged

13. UX principles and design system

The interface stays calm and simple while the intelligence handles the complexity. The goal is under 3 taps (or one sentence) from opening the app to the first cooking step.

Principles

1. **One answer, not a list.** One hero pick with a reason; alternates are small.
2. **Value before data.** A useful pick on day one from three onboarding answers.
3. **Ask the fewest, highest-value questions.** Only ask what changes tonight's plan.
4. **Explain lightly.** One plain reason, never model jargon.
5. **Cooking is physical.** Big type, big targets, voice-first in Cook, forgiving of mistakes.
6. **The user stays in control** of purchases, memory and pantry changes.
7. **Trust over magic.** Show uncertainty for scans, expiry and freshness.
8. **Don't over-card.** Keep the airy, premium feel; not every element is a card.

Design tokens ("Morning kitchen light")

This warm palette replaces the blueprint's lavender-led pastel direction, which reads as generic AI styling; it needs design-lead approval.

Token	Light	Dark	Use
`color.bg` (Rice Cream)	#FAF6EF	#1C1A17	App background
`color.surface` (Ghee)	#F3E9D6	#26231F	Cards, sheets
`color.text` (Masala Charcoal)	#2B2A28	#F1EBE0	Primary text
`color.textMuted` (Warm Stone)	#6B655C	#B8AFA2	Secondary text
`color.primary` (Tomato)	#E5533D	#EE6A55	Main action; small text uses #C83F2A for contrast
`color.highlight` (Haldi)	#F2B33D	#F5C25E	Use Soon, highlights
`color.success` (Curry Leaf)	#3F7D58	#5FA67B	Done, healthy nudges
`color.tint.*`	#FCEFD2, #FBE3DE, #E3F0E7, #E2ECF4	12% of the accents	Calm variety; sky tint for cold storage

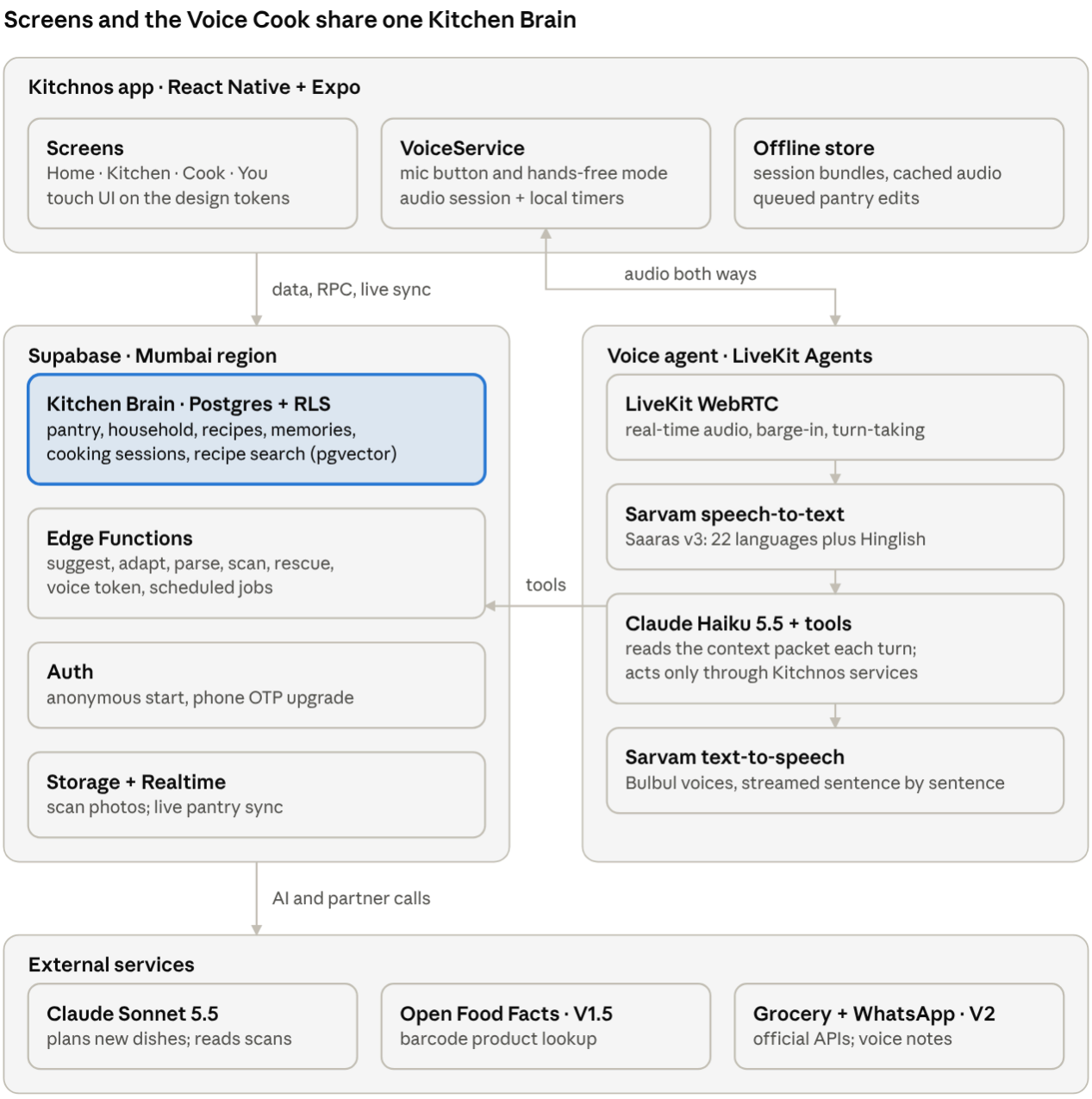
- **Type:** a serif for headings (Fraunces) and a clean sans for body (Inter); Devanagari and regional scripts use Noto families matched for weight.
- **Shape:** 24 px radius on large surfaces, 12 px on chips; soft shadows; no gradients, glows, sparkles or robot icons.

- **Motion:** soft springs, tactile selection, ingredient-added motion, calm step transitions; all reduced under the system's reduce-motion setting.
- **Rule:** one strong accent per screen; let food photography carry most of the colour.

Screen inventory: Onboarding (3), Home, Craving sheet, Kitchen (4 locations), Item detail, Add/confirm, Recipe detail, Why this works, Cook step, Photo checkpoint, Something's wrong, Rescue fix, Finish and feedback, You, What I've learned, Shopping list, plus loading, empty, offline and error states for each.

14. System architecture

The app and the voice agent are two clients of the same Kitchen Brain. The app talks to Supabase directly for data and through Edge Functions for AI work; the voice agent receives audio over WebRTC and acts only through the same RPCs and Edge Functions, using the signed-in user's token so row-level security applies to voice exactly as to touch.



system architecture · app, Kitchen Brain, voice agent

Only the Kitchen Brain is highlighted because it is the one place state lives; everything else can be swapped behind its service boundary. Component-level detail (hosting, versions, dependencies) is in `01_system_components.csv`; the schema in `02` – `06`.

Key technology choices

Layer	Choice	Why
Mobile	React Native + Expo (SDK 58) + TypeScript, Expo Router, development build	Blueprint stack; real-time audio needs native modules

Layer	Choice	Why
Client state	TanStack Query for server state, Zustand for UI and session state, SQLite + MMKV for offline	Clean split between cache and live cooking state
Backend	Supabase: Postgres 15+, row-level security, Edge Functions (Deno), Realtime, Storage, pg_cron, pgvector	One managed backend, Mumbai region
Voice transport	LiveKit Cloud WebRTC + `@livekit/react-native`	Echo cancellation, barge-in, low latency on mobile networks
Voice agent	LiveKit Agents 1.8 (Python) with Sarvam, Anthropic and Silero plugins, self-hosted workers	Cost (BRD section 8) and control
AI	Claude Haiku 5.5 (turns, parsing), Claude Sonnet 5.5 (planning, vision)	Speed for voice; quality for plans
Speech	Sarvam Saaras v3 (speech-to-text), Bulbul (text-to-speech)	22 Indian languages, code-mixed speech
Observability	Sentry, PostHog, per-turn voice telemetry table	Latency and cost gates
Delivery	EAS Build and Update, GitHub Actions	Blueprint's GitHub workflow

15. Analytics and success metrics

Every journey is instrumented from day one (BR-20). The full event dictionary, with properties and the KPI each event feeds, is `12_analytics_events.csv`.

KPI	Definition	Beta target	Events
Pick acceptance	Picks accepted ÷ picks shown	≥ 40%	`pick_shown`, `pick_accepted`, `pick_dismissed`
Cooking frequency	Completed sessions per active household per week	≥ 3	`cook_started`, `cook_completed`

KPI	Definition	Beta target	Events
Activation	New households completing a guided meal within 7 days	⩾ 40%	`onboarding_completed`, `cook_completed`
Hands-free rate	Voice sessions completed with no taps after start	⩾ 60%	`voice_session_started`, `cook_completed` (taps_after_start)
Voice latency	End of speech to first audio, per turn	p50 ⩽ 1.0 s	`voice_turn` (latency_ms)
Pantry trust	Corrections per household per week	⩽ 5	`inventory_corrected`
Waste rescue	Use Soon items cooked before expiry	⩾ 50%	`use_soon_shown`, `inventory_consumed`
Retention	Households active on day 30	⩾ 25%	`app_open`
Cost	Variable cost per voice session	⩽ ₹5	`voice_session_ended` (cost_inr)
Rescue success	Sessions completed after a rescue	⩾ 80%	`rescue_requested`, `cook_completed`

Analytics never include raw transcripts, recipe free text or pantry contents beyond counts; events carry household and member IDs only.

16. Release plan and acceptance criteria

V1 is built in six two-week sprints following the blueprint (dates in the BRD roadmap). Each sprint ends with a demo on a real phone in a real kitchen.

Sprint	Scope	Done when
S1 Shell + Home	Navigation, design tokens, primitives, Home with mock picks and all states; Supabase project, schema migration, auth (anonymous + phone OTP)	Home renders all states on a 3 GB Android; migration applies cleanly

Sprint	Scope	Done when
S2 Kitchen	Locations, item detail, add/edit/finish, natural-language add, staples, components	"2 kilo aloo, 1 dozen ande" becomes two confirmed items
S3 Craving + recipes	Craving input, ranking, recipe detail, pantry match, adaptation, family plates, curated base (first 100 dishes)	A pick uses real pantry items and states why
S4 Cook With Me + Voice	Steps, timers, checkpoints, rescue, completion; LiveKit voice agent, Sarvam speech, fast-path commands, hands-free mode	A dish is cooked start to finish by voice only
S5 You + learning	Feedback, memories with Edit · Keep · Forget, taste profiles, "less spicy, as you asked"	"Agli baar lahsun zyada" shows on You and changes the next recipe
S6 Backend integration + hardening	Mocks replaced by services, offline bundles, retention jobs, analytics, cost telemetry, 300 dishes	The full first-magic-moment loop passes on 5 devices

V1 release acceptance (all must pass):

- [ ] The 9-step first magic moment (section 1) passes by touch and by voice in Hindi, Hinglish and English.
- [ ] Five rural pilot users finish a dish by voice without help with the phone.
- [ ] Voice latency p50 ⩽ 1.0 s and p95 ⩽ 1.8 s over 200 logged turns.
- [ ] Cook mode completes in airplane mode after start.
- [ ] Row-level security tests pass for every table; no service key in the app bundle.
- [ ] Consent, data export and deletion work end to end.
- [ ] Variable cost per voice session measured at ⩽ ₹5 over the beta.
- [ ] BRD section 3 beta gates met or a documented decision to extend the beta.

Later releases follow the blueprint: V1.5 frictionless input (barcode, camera, receipt; weekly plate), V2 Kitchen Agent (leftovers, predictive restock, grocery integrations, WhatsApp, paid tier, cook handoff), V3 Household Food OS (approved auto-replenishment, provider comparison, phone-call channel, family recipe capture).

17. Dependencies, risks and open questions

External dependencies and the full risk register are in BRD sections 12 and 13. The product risks that most shape V1 design:

- **Pantry drift** → probable inventory with confidence, deduction after cooking, one targeted question a day.
- **Dialect and noise errors** → Indic speech model, on-device voice detection, repeat-back confirmation, field tests per dialect.
- **Unsafe AI recipes** → curated foundations plus variation rules; safety rules the model cannot override.
- **Voice cost** → the four cost controls in VOICE-14, measured per session from day one.

Open questions

- [ ] Launch regional language and its dialect test set.
- [ ] Who curates and photographs the first 300 dishes, and by when?
- [ ] Free-tier voice cap (BRD section 8).
- [ ] Approve the warm design palette over the blueprint's pastel direction.
- [ ] Anonymous accounts: how long do unclaimed households live before cleanup (proposed 180 days)?
- [ ] Which rural pilot partner, and how many devices do we lend for the pilot?
- [ ] Is a separate cook persona with her own voice profile needed in V1, or V2 as planned?

18. Appendix: architecture CSV pack and GitHub setup

The architecture is delivered as CSV files so engineers can sort, filter and import it; the Supabase migration and TypeScript types in the repo are generated from the same schema definition, so they cannot drift apart.

File	Contents
`01_system_components.csv`	Every component: layer, technology, responsibility, hosting, dependencies
`02_database_tables.csv`	Every table: domain, purpose, owning service, access pattern, retention, release
`03_database_columns.csv`	Every column: type, nullability, default, keys, constraints, index, description

File	Contents
`04_relationships.csv`	Foreign keys: cardinality and on-delete behaviour
`05_rls_policies.csv`	Row-level security policy per table and operation
`06_storage_buckets.csv`	Storage buckets, paths, access and retention
`07_voice_agent_tools.csv`	Voice tools: parameters, service called, confirmation rule, example phrase
`08_api_contracts.csv`	Service operations: transport, endpoint, request, response, tables touched, caller
`09_data_flows.csv`	Step-by-step data flow for every journey
`10_functions_and_jobs.csv`	Postgres RPCs, Edge Functions and scheduled jobs
`11_ai_tasks_and_models.csv`	Each AI task: model, inputs, output schema, latency budget, fallback, evaluation
`12_analytics_events.csv`	Event dictionary with properties and KPI mapping
`13_env_and_secrets.csv`	Environment variables and where each secret lives
`14_user_stories.csv`	User stories with acceptance criteria, priority, sprint, labels, estimates
`15_epics.csv`	Epics with goals and linked requirements
`16_github_labels.csv` / `17_github_milestones.csv` / `20_github_project_fields.csv`	GitHub labels, milestones and Project fields
`18_repo_structure.csv`	Every folder in the monorepo and what goes there
`19_requirements_traceability.csv`	BR → PRD requirement → user stories → tables

GitHub setup. The `kitchnos` monorepo ships with issue templates, pull-request template, code owners, CI workflows, the generated Supabase migration, an Expo app skeleton, the voice agent skeleton and a script that creates labels, milestones, every user

story as an issue and a GitHub Project board in one run.